1. Configuración y uso básico de GitHub

Implementar el control de versiones sería una tarea sencilla si sólo trabajaras como desarrollador en solitario con un repositorio local. Sin embargo, este no es el caso de muchos proyectos profesionales. En su lugar, un equipo agrupará recursos en un repositorio remoto utilizando un alojamiento como GitHub. Por ello, es importante aprender a enviar a GitHub porque es algo que tendrás que hacer como parte de tu flujo de trabajo.

GitHub

GitHub es una plataforma donde puedes almacenar, compartir y trabajar junto con otros usuarios para escribir código.

Almacenar tu código en un "repositorio" en GitHub te permite:

- **Presentar o compartir** el trabajo.
- **Seguir y administrar** los cambios en el código a lo largo del tiempo.
- Dejar que otros usuarios **revisen** el código y realicen sugerencias para mejorarlo.
- **Colaborar** en un proyecto compartido, sin preocuparse de que los cambios afectarán al trabajo de los colaboradores antes de que esté listo para integrarlos. El trabajo colaborativo, una de las características fundamentales de GitHub, es posible gracias al software de código abierto Git, en el que se basa GitHub.

Existen también otros hosts VCS online, como GitLab, BitBucket, Buddy, etc.

Para empezar a trabajar con GitHub, deberá crear una cuenta personal gratuita y comprobar la dirección de correo electrónico.

Cada persona que utilice GitHub.com deberá iniciar sesión en una cuenta personal. Tu cuenta personal es tu identidad en GitHub.com y tiene un nombre de usuario y perfil.

1. Ir a https://github.com/.
2. Hacer clic en **Registrarse**.
3. Seguir las indicaciones para crear tu cuenta personal.

Durante el registro, se te pedirá que verifiques tu dirección de correo electrónico.
Se recomienda encarecidamente configurar 2FA para tu cuenta. 2FA es una capa adicional de seguridad que puede ayudar a mantener tu cuenta segura. (Nos descargamos en el móvil Microsoft Authenticator para doble factor de autenticación y desde ahí escaneamos el código qr de la aplicación.)

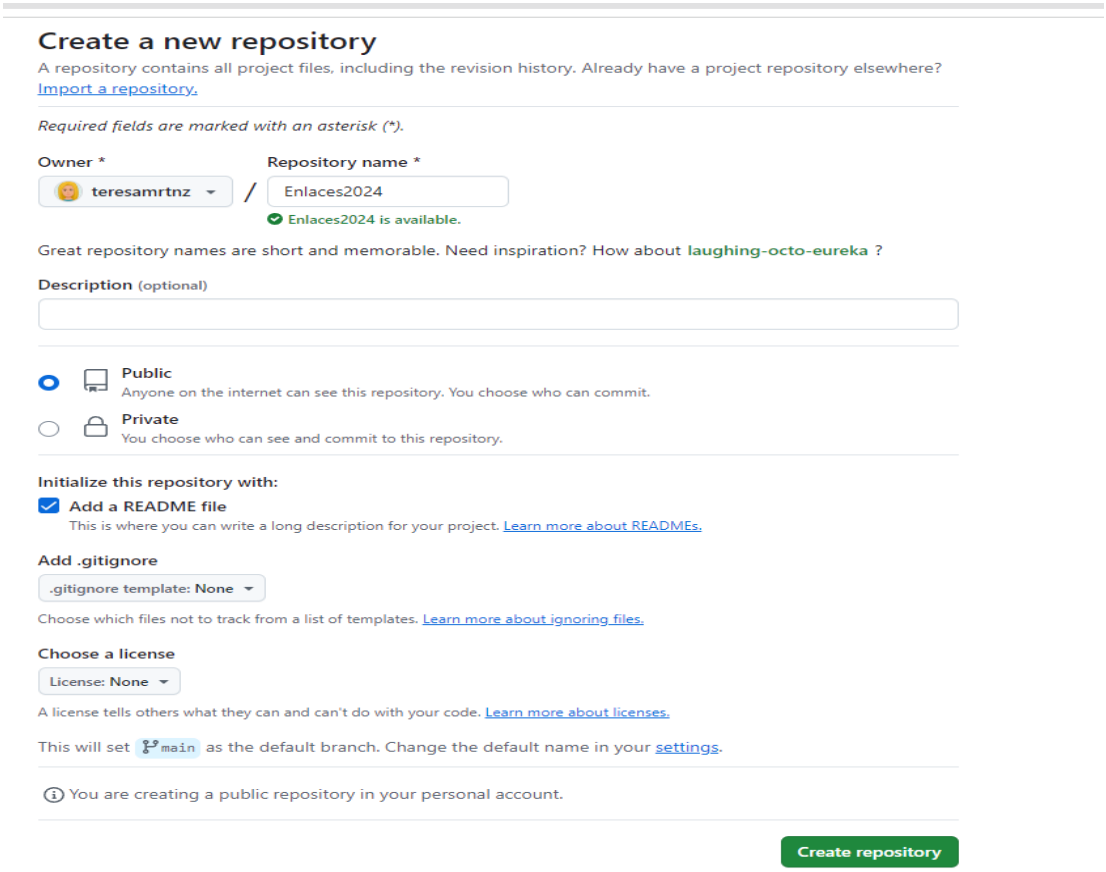
1. Primeros pasos en Github

Creación de un repositorio, hacer cambios y commitearlos

El primer paso es crear un nuevo repositorio online dentro de GitHub.
Un repositorio es el elemento más básico de GitHub. Es un lugar donde puedes almacenar el código, los archivos y el historial de revisiones de cada archivo. Los repositorios pueden contar con múltiples colaboradores y pueden ser públicos como privados.

Para crear un repositorio en GitHub, solo hay que seleccionar el botón + de la barra de herramientas y "new repository".

Seleccionar la opción de inicializar el repo con README. Este fichero se suele usar para describir el proyecto.



Modifica el fichero README.md (desde el icono lápiz). Navega hasta la parte inferior e introduce un mensaje de commit que describa tu aportación para realizar el primer commit.

Clonando repositorios

Se puede clonar un proyecto de github para poder modificarlo en local. Un clon es una copia completa de un repositorio, incluidos todos los registros y versiones de los archivos.

Desde un repositorio original en github haz clic en el botón verde "Código" para obtener la URL HTTPS a clonar. Desde el terminal hay que posicionarse en el directorio en el que quieres copiar el proyecto y ejecutar `git clone url`. Echa un vistazo a tu sistema de archivos y verás un nuevo directorio llamado como el proyecto clonado.

```
lenovo@LAPTOP-K87IDOM2 MINGW64 /d/LosEnlaces20242025/Despliegue/UD2 GIT
$ git clone https://github.com/teresamrtnz/examenDespliegue.git
Cloning into 'examenDespliegue'...
remote: Enumerating objects: 11, done.
remote: Total 11 (delta 0), reused 0 (delta 0), pack-reused 11 (from 1)
Receiving objects: 100% (11/11), done.
Resolving deltas: 100% (1/1), done.

lenovo@LAPTOP-K87IDOM2 MINGW64 /d/LosEnlaces20242025/Despliegue/UD2 GIT
$ ls
ejercicio1/  examenDespliegue/    repositorioGit/
ejercicio2/  practicaEjercicio1/
```

Mostrando tus repositorios remotos

Para ver qué repositorios remotos tienes configurados, puedes ejecutar el comando

```
git remote
```

Mostrará una lista con los nombres de los remotos que hayas especificado. Si has clonado tu repositorio, deberías ver por lo menos "origin", es el nombre predeterminado que le da Git al servidor del que clonaste.

También puedes añadir la opción -v, que muestra la URL asociada a cada repositorio remoto.

```
git remote -v
```

2. Repositorios remotos propios

Descargar repositorios remotos y sincronizar cambios

Para poder colaborar en cualquier proyecto Git, necesitas saber cómo gestionar tus repositorios remotos. Los repositorios remotos son versiones de tu proyecto que se encuentran alojados en Internet o en algún punto de la red. Puedes tener varios, cada uno de los cuales puede ser de sólo lectura, o de lectura/escritura, según los permisos que tengas. Colaborar con otros implica gestionar estos repositorios remotos, y mandar (push) y recibir (pull) datos de ellos cuando necesites compartir cosas.

Añadiendo repositorios remotos

Lo primero que vamos a hacer es añadir el repositorio remoto. Para añadir un nuevo repositorio Git remoto, asignándole un nombre con el que referenciarlo fácilmente, ejecuta

```
git remote add [nombre] [url]
```

```
$ git remote
origin
$ git remote add pb https://github.com/paulboone/ticgit.git
$ git remote -v
origin  https://github.com/schacon/ticgit.git
pb  https://github.com/paulboone/ticgit.git
```

Ahora puedes usar la cadena "pb" en la línea de comandos, en lugar de toda la URL.

Recibiendo actualizaciones de repositorios remotos

Una vez añadido el repo remoto, puedes recuperar datos de tu repositorio remoto ejecutando:

```
$ git fetch [remote-name] [branch]
```

Es importante tener en cuenta que el comando fetch sólo recupera la información y la pone en tu repositorio local en una rama oculta (repo/rama) que puedes ver con `git branch -a`, no la une automáticamente con tu trabajo ni modifica aquello en lo que estás trabajando.

```
PS C:\Users\lenovo\Downloads\ejercicio2> git fetch origin main
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 994 bytes | 58.00 KiB/s, done.
From https://github.com/teresamrtnz/ejercicio2
 * branch            main       -> FETCH_HEAD
   6c5283c..a859b6d  main       -> origin/main
PS C:\Users\lenovo\Downloads\ejercicio2> git log --oneline --all
a859b6d (origin/main) Update ejercicio2.html
6c5283c Update ejercicio2.html
bee5516 (HEAD -> main) version final
518ac02 Update ejercicio2.html
44ebeae mergeado
c0f6d42 (feature1) otra funcionalidad
baadc86 (hotfix) hecho el apaño
610d68e añadida imagen
4dc580d primer commit
```

Por eso después del fetch tendrías que ejecutar git merge si quisieras unirla con tu trabajo.

```
PS C:\Users\lenovo\Downloads\ejercicio2> git branch
  feature1
  hotfix
* main
PS C:\Users\lenovo\Downloads\ejercicio2> git merge origin/main
Updating bee5516..a859b6d
Fast-forward
 ejercicio2.html | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
```

```
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2$ cd proyectoGit/
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2/proyectoGit$ ls
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2/proyectoGit$ git remote add enlaces2024 https://github.com/teresamrtnz/Enlaces2024.git
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2/proyectoGit$ ls
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2/proyectoGit$ ls -la
total 12
drwxrwxr-x 3 alumno alumno 4096 oct 16 23:12 .
drwxrwxr-x 3 alumno alumno 4096 oct 16 23:11 ..
drwxrwxr-x 7 alumno alumno 4096 oct 17 00:31 .git
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2/proyectoGit$ git remote -v
enlaces2024     https://github.com/teresamrtnz/Enlaces2024.git (fetch)
enlaces2024     https://github.com/teresamrtnz/Enlaces2024.git (push)
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2/proyectoGit$ git fetch enlaces2024 master
fatal: no se pudo encontrar ref remota master
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2/proyectoGit$ git fetch enlaces2024 feature1
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Desempaquetando objetos: 100% (9/9), 2.58 KiB | 2.58 MiB/s, listo.
Desde https://github.com/teresamrtnz/Enlaces2024
 * branch            feature1   -> FETCH_HEAD
 * [nueva rama]      feature1   -> enlaces2024/feature1
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2/proyectoGit$ git merge
fatal: No hay remoto para la rama actual.
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2/proyectoGit$ git merge enlaces2024/feature1
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2/proyectoGit$ ls
README.md
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2/proyectoGit$ cat README.md
# Enlaces2024 prueba pruebaenFeature
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2/proyectoGit$
```

El comando

`git pull [remote-name] [branch]`

se usa para recuperar y unir automáticamente la rama remota con tu rama actual (hace un `git fetch [remote-name] [branch]` + `git merge [remote/brach]`). Éste puede resultarte un flujo de trabajo más sencillo y más cómodo.

<u>Enviando a tu repositorio remoto</u>

Cuando tu proyecto se encuentra en un estado que quieres compartir, tienes que enviarlo a un repositorio remoto. El comando que te permite hacer esto es sencillo:

`git push [nombre-remoto][nombre-rama]`

Si quieres enviar tu rama maestra (master) a tu servidor origen (origin), ejecutarías esto para enviar tu trabajo al servidor:

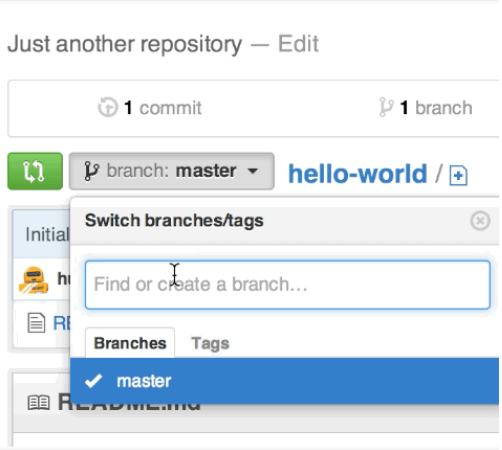
```
git push origin master
```

Este comando funciona únicamente si has clonado de un servidor en el que tienes permiso de escritura, y nadie ha enviado información mientras tanto. Si tú y otra persona clonais a la vez, y él envía su información y luego envías tú la tuya, tu envío será rechazado. Tendrás que bajarte primero su trabajo e incorporarlo en el tuyo para que se te permita hacer un envío.

3. Ramas en GitHub

Creación de una rama

Github permite la creación de ramas haciendo click en el botón branch:master y escribiendo el nombre de la nueva rama "feature1" nos creará y moverá a la nueva rama.

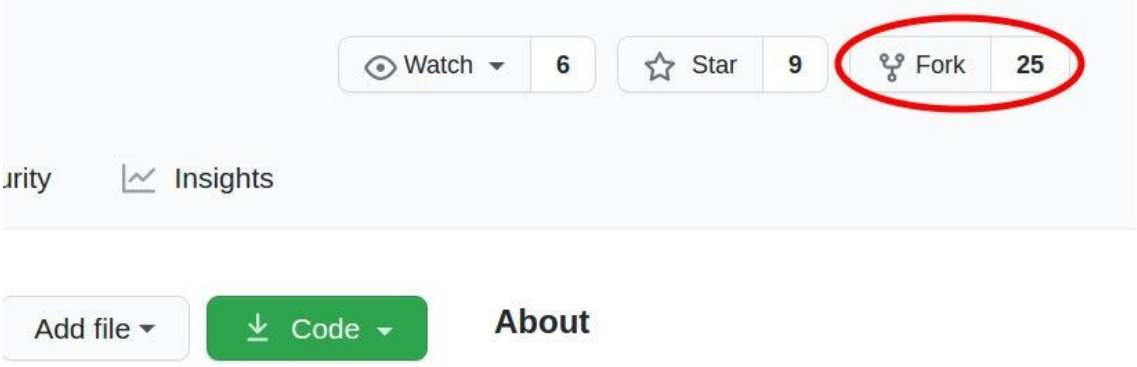


Ahora puedes hacer de nuevo cambios en la nueva rama y commitearlos.

4. Contribuyendo en repositorios remotos de otros (fork y pull request)

Cuando nos gusta el repositorio de alguien y nos gustaría tenerlo en nuestra cuenta de GitHub, hacemos un fork o bifurcación para poder trabajar con él en forma separada. Cuando hacemos un fork de un repositorio, obtenemos una instancia de todo el repositorio con todo su historial. Luego, podemos hacer lo que queramos sin afectar la versión original.

Para realizar un fork del repositorio hay que hacer clic en el botón fork de la parte superior de la página. Esto creará una instancia del repositorio completo en tu cuenta.



Una vez que el repositorio esté en tu cuenta, puedes clonarlo a tu ordenador para trabajarlo localmente.

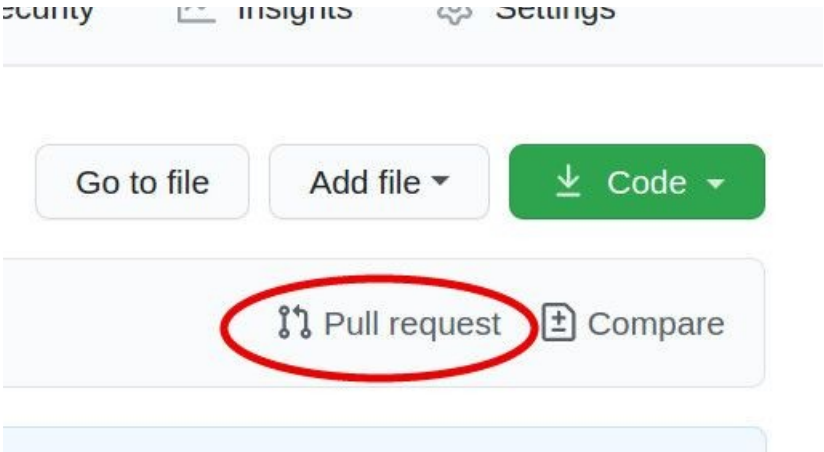
Es una buena práctica crear una rama nueva cuando trabajas con repositorios, el nombre de la rama debe ser breve y debe reflejar el trabajo que estamos haciendo.

Una vez has creado la rama y hecho cambios debes commitearlos para luego enviar tus cambios a Github.

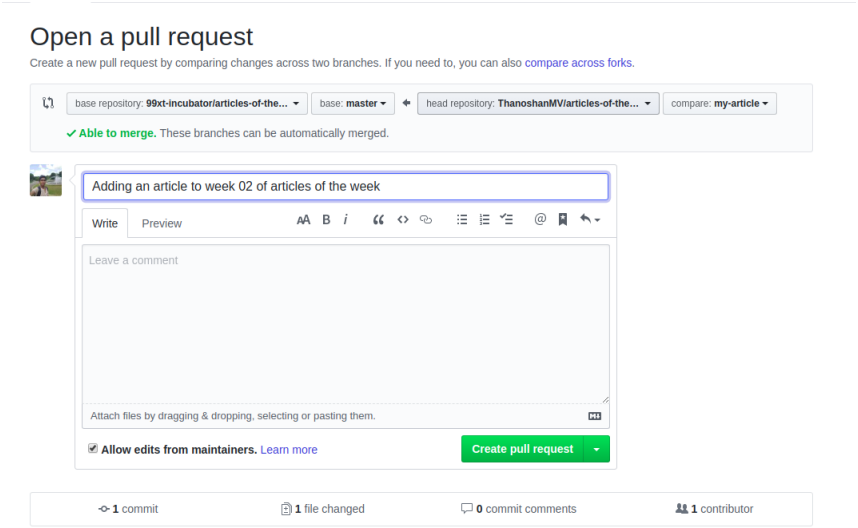
Para enviar los cambios a GitHub, debemos identificar el nombre del repositorio remoto. Con el comando push subiremos la rama creada a github.

```
git push origin [Nombre de la Rama]
```

Ahora puedes querer avisar al desarrollador origen de los cambios que has realizado para que pueda agregarlos a su proyecto y de esa forma poder contribuir a un proyecto de otro desarrollador. El otro desarrollador podrá aceptar o declinar el ofrecimiento. Es lo que se denomina “PULL REQUEST”.



Por ejemplo, un usuario llamado Harry realiza un fork de un repositorio de ThanoshanMV y le efectúa algunos cambios. Ahora Harry puede hacer un pull request a ThanoshanMV, pero dependerá de ThanoshanMV aceptar o declinarlo. Si tu pull request es aceptado recibirás un mail.



Antes de enviar cualquier pull request deberías sincronizar tu repositorio, ya que el otro programador puede haber agregado algunas prestaciones o funciones adicionales y haberse corregido algunos errores desde la vez que realizaste un fork de aquel repositorio.

1. Primero, revisa en que rama estás ubicado.

$ git branch

Esto enumerará todas las ramas y indicará en verde la rama actual o activa.

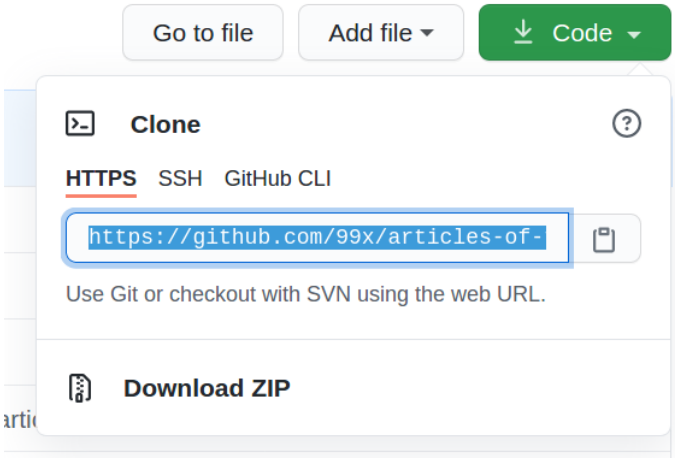
2. Cambia a la rama maestra.

$ git checkout master

3. Agrega el repositorio original como un repositorio upstream.

Para poder extraer los cambios desde el repositorio original a tu versión local, necesitas agregar el repositorio Git original como un repositorio remoto de nombre upstream.

$ git remote add upstream [HTTPS]



4. Busca (fetch) el repositorio.

Busca todos los cambios del repositorio original. Las confirmaciones (commits) del repositorio original serán almacenadas en una rama local llamada upstream/master.

$ git fetch upstream

5. Fusionala.

Fusiona los cambios de la rama upstream/master a tu rama maestra local. Esto hará que tu rama maestra se sincronice con el repositorio upstream sin perder tus cambios locales.

$ git merge upstream/master

6. Envía (push) los cambios a GitHub

En este punto tu rama local está sincronizada con la rama maestra del repositorio original. Si deseas actualizar el repositorio de GitHub, necesitas enviar tus cambios.

$ git push origin master

NOTA: Luego de sincronizar tu rama maestra puedes eliminar el repositorio upstream, si lo desea. Pero lo necesitará para actualizar/sincronizar tu repositorio en el futuro, por lo que es una buena práctica conservarlo.

$ git remote rm [Nombre del Repositorio Remoto]

8. Elimina ramas innecesarias

Las ramas son creadas para propósitos especiales. Una vez que ese propósito se cumple, aquellas ramas ya no son necesarias, por lo que puedes eliminarlas.

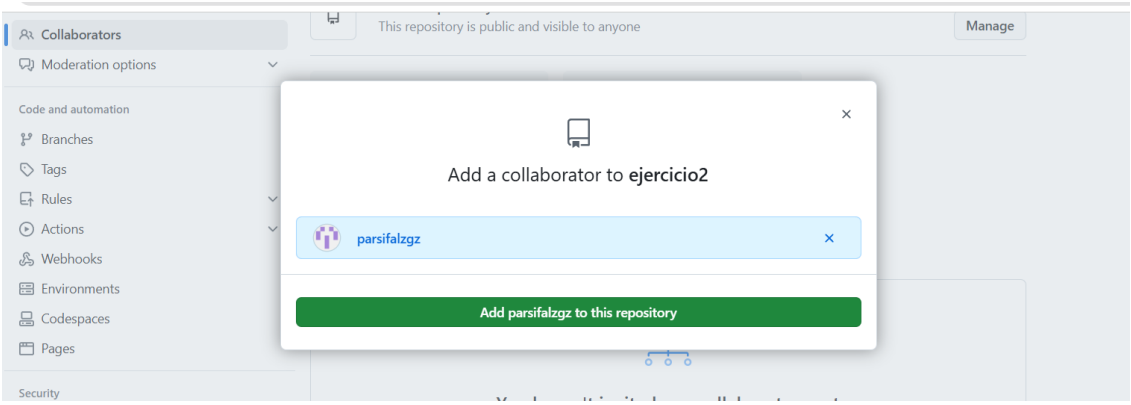
$ git branch -d [Nombre de la Rama]

También, puedes eliminar su versión en GitHub.

git push origin --delete [Nombre de la Rama]

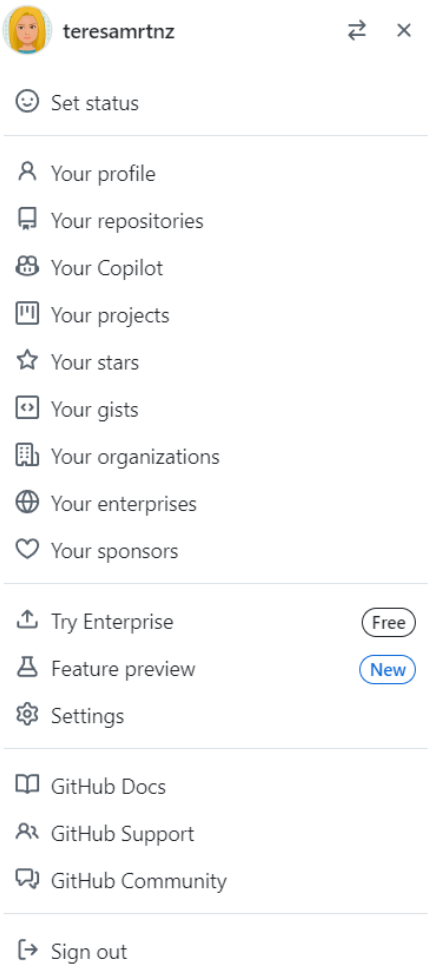
5. Añadiendo colaboradores en un repositorio de Github

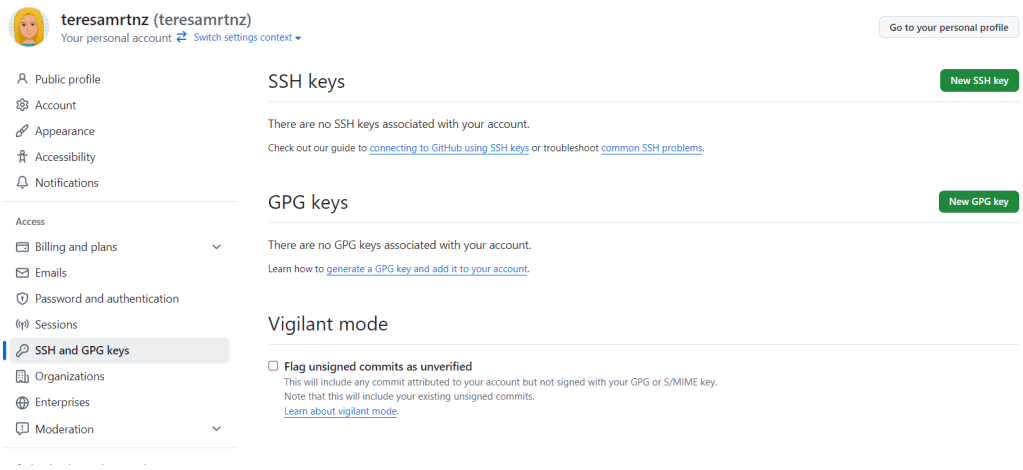
La fortaleza de Git y GitHub se encuentra en la colaboración para el desarrollo de software. En esta lección aprenderemos cómo añadir colaboradores en un repositorio remoto alojado en GitHub. Para añadir colaboradores a un repositorio remoto al que pertenecemos alojado en GitHub, accedemos a la URL del repositorio en cuestión y, habiendo iniciado sesión, hacemos clic en Settings y a continuación seleccionamos la opción Colaborators. Probablemente seamos redirigidos a una vista de confirmación de acceso en la que puede que nos pidan, por cuestiones de seguridad, la introducción de un código de activación que recibiremos por mensaje de texto. Vamos a pedir que nos envíen ese mensaje de texto y lo introducimos en este cuadro de texto. Tras introducir dicho código, ahora sí, podremos gestionar el acceso de colaboradores al repositorio. Hacemos clic sobre el botón Add people y a continuación introducimos el nombre de usuario o el correo electrónico de la persona que deseamos añadir.



6. Acceso mediante SSH (clave)

Si en vez de acceder con la URL (introduciendo usuario y contraseña) queremos acceder al repositorio remoto mediante SSH hay que hacerlos mediante un par de claves. Vamos a los settings y asociamos una ssh-key.





Para crear nuestra ssh-key ejecutamos el comando ssh-keygen en la terminal del ordenador y copiaremos el contenido de la clave pública a una nueva clave ssh en GitHub

```
alumno@alumno-VirtualBox:~/Documentos/despliegue2425/ud2/proyectogit$ cd
alumno@alumno-VirtualBox:~$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/alumno/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/alumno/.ssh/id_ed25519
Your public key has been saved in /home/alumno/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:i+DkWWiaapBx2xmCEcm0dy+wEUBqjTiaSM5s2L0QS0k alumno@alumno-VirtualBox
The key's randomart image is:
+--[ED25519 256]--+
|=*E.             |
|=o+..            |
|+B+= .           |
|%=++*..          |
|=O+=*+..S        |
|+ .Oo=.. .       |
|. o = . .        |
| o               |
|o                |
+----[SHA256]-----+
```

SSH keys

New SSH key

This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.

Authentication keys

Portátil personal

SHA256:i+DkWWiaapBx2xmCEcm0dy+wEUBqjTiaSM5s2L0QS0k

SSH

Added on Oct 17, 2024

Never used — Read/write

Delete

Check out our guide to connecting to GitHub using SSH keys or troubleshoot common SSH problems.

7. Práctica3